

Project-2: Interacting with the Physical World using Raspberry Pi

Names: Sualeh 10303, Abdullah 10317

Goals

- Get familiar with use of Raspberry Pi for input and output.
- Interfacing RGB LEDs, keypads, buzzers, and ultrasonic sensors with the Raspberry Pi.
- Practice using the API documentation of a library, specifically GPIOZero, for the purpose of building solutions.

1 Introduction

Our previous project was about effective energy transfer, i.e. how can we transfer electrical power effectively from the battery to the LED and convert it to light. We know that electricity can also be used for information transfer, processing, and storage. How? Say you wanted to communicate the current temperature of the room to your friend. You could both agree on a correspondence between each temperature value and a voltage value. Communicating the temperature then essentially becomes a question of transferring that corresponding voltage value to your friend, and you can design a circuit to do that. This project will highlight this alternate facet of the use of electricity.

1.1 Analog vs Digital

Information can be represented in either analog form or digital. Consider the same example of the temperature of a room. In Figure 1, we see two thermometers showing the temperature in the room. The analog thermometer has a continuous output as the temperature reading can fall anywhere between the marks on the tube. Every temperature reading is fair game. In contrast, if the current reading on the digital thermometer is 21.8° and the temperature increases, we know that the next reading we'll see will be 21.9° , as 0.1 is the smallest quantity that can be shown on this thermometer. So, the output of this thermometer is discrete or it can only assume some values.

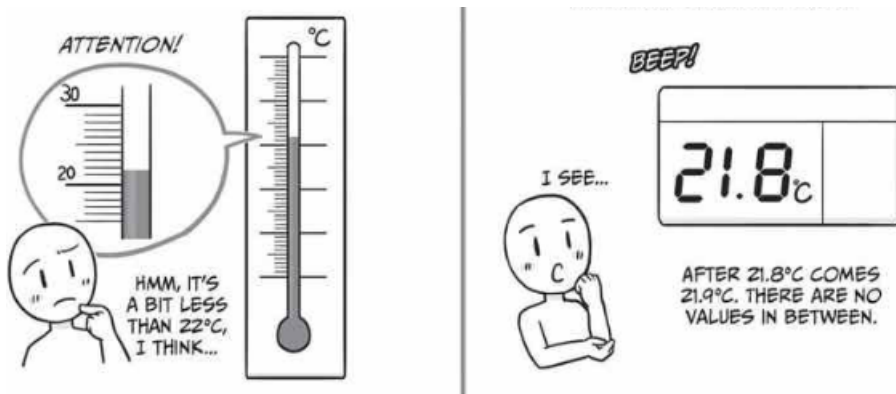


Figure 1: Analog and Digital Thermometers. Source: Shibuya, M., & Tonagi, T. (2017). The Manga Guide to Microprocessors.



A signal is a function of time that carries information. An **Analog** signal's output is continuous and it can assume any possible value. A **Digital** signal's output is discrete and it can only assume values from a defined set of numbers. Each value of a digital or analog signal corresponds to some information.

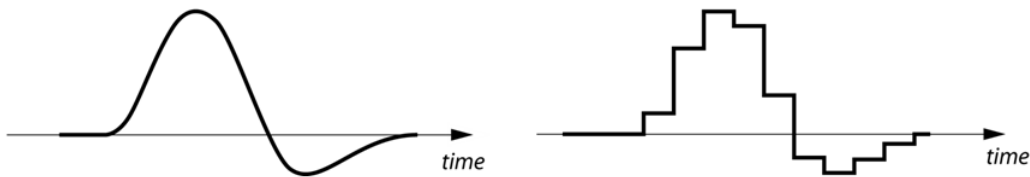


Figure 2: **Left:** Analog Signal. **Right:** Digital Signal. Note that distinction between analog and digital is being made on the basis of the output. The input, time, in both cases is continuous.

1.2 Logical Values or States

We're already familiar with some digital signals, such as the square pulse from the previous project. There were two possible states for the square pulse: 'HIGH' or 'LOW'; if the voltage was close to 9V, it was considered HIGH and if it was close to 0V, then it was interpreted as LOW. This is an example of a binary (two possible states) digital signal. Recall that the two states 'HIGH' and 'LOW' carried the information that the LED is 'ON' or 'OFF' respectively. *Does every digital signal have only two possible states?*

No. However, our modern digital systems are binary systems.



Binary digital signals have only two logical values or states, often distinguished as 'one' and 'zero'; 'HIGH' and 'LOW'.

Does this mean that we can only represent two pieces of information with binary digital signals?

We listen to entire songs digitally, so this certainly cannot be the case. We increase our information carrying capacity in digital systems by grouping multiple digital signals.



We can represent any analog quantity if we group enough individual binary digital signals ("binary digits or bits") together.

If we use two bits, we have increased our information carrying capacity to four. Each of the bits can take on two values, so there are four possible combinations of values for the two bits together. Each combination can be associated with a different piece of information.

1.3 Which is better - digital or analog?

You could argue that an analog signal has better information carrying potential as each value can be assigned a different information correspondence. Then, why are most present systems digital when digital signals, by definition, have limited number of states? It is due to the high susceptibility of analog signals to noise, i.e. the information can be easily corrupted. Say voltage signal is carrying information. If the voltage changes from say 5V to 5.1V, our information is corrupted, since both these voltage values correspond to different information.



Digital systems are more resilient to noise, offer an ease of computation, and are typically cheaper, compared to analog systems. The trade off is the additional complexity of digital systems.

Why are digital systems resilient to noise? We'll answer this question in a later section, today.

2 Getting to know the Raspberry Pi

Raspberry Pi is a credit-card-sized, affordable computer. The creator's goal behind the Pi was to create a cheap computer in order to get kids started with computers and programming earlier. It might be interesting to hear the history <https://youtu.be/UCt6d0SCx04> directly from its creator, Eben Upton. You can also see some interesting developments in the past few years by watching '10 years of Raspberry Pi' (<https://youtu.be/3-QAUgtmVok>). The official documentation for Raspberry Pi can be located at <https://www.raspberrypi.com/documentation/>.

We're going to use Raspberry Pi 3 Model B, Raspberry Pi 3 Model B+, or Raspberry Pi 4 Model B in the lab. The location of the main ICs and connectors on Raspberry Pi 3 and Pi 4 are shown in Figure 3 and Figure 4, respectively. Detailed specifications of the different Raspberry Pi products can be found at <https://www.raspberrypi.com/products/>.

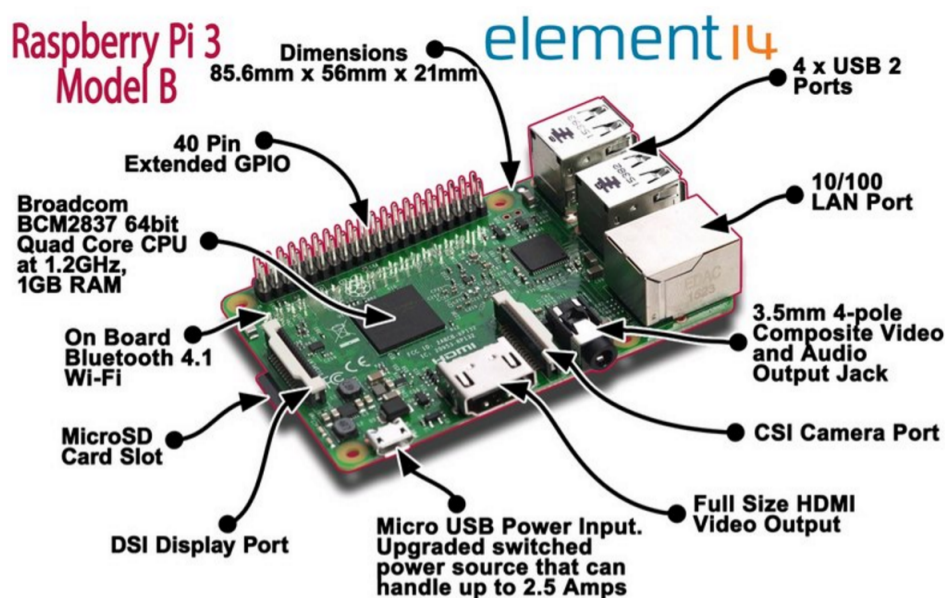


Figure 3: Raspberry Pi 3

2.1 Raspberry Pi doesn't look like a computer. What is a computer?



The basic structure of any computer is shown in Figure 5. A stored program computer is a device that performs the tasks stored as a program on its memory. It does so by continuously repeating a cycle of fetching instruction from its instruction memory, decoding the instruction, and then executing it.

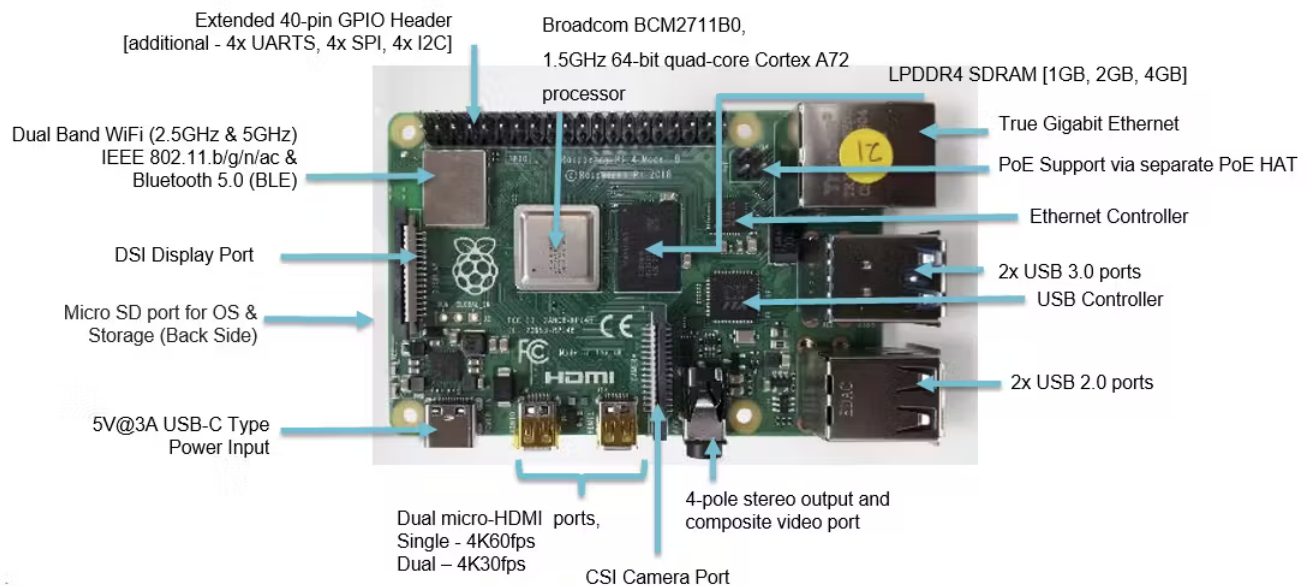
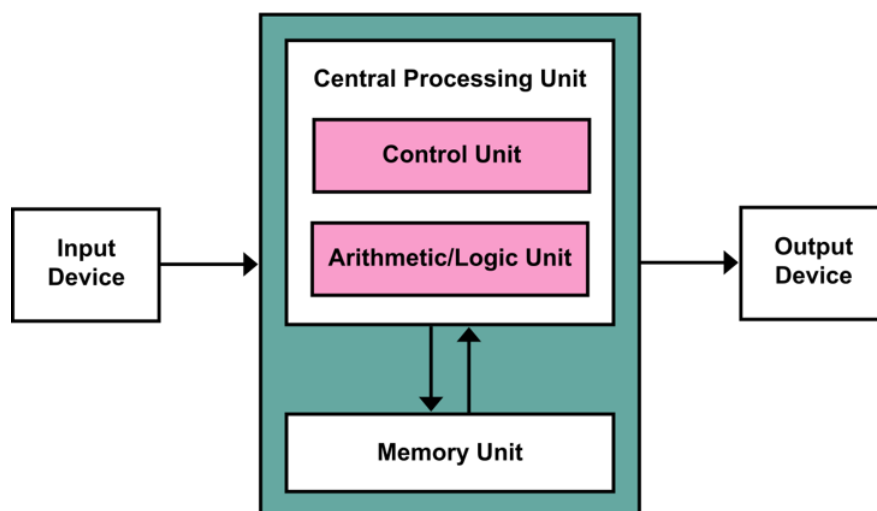


Figure 4: Raspberry Pi 4

Figure 5: Stored Program Computer.¹

The program is stored on the memory in a language that the particular computer hardware is wired

¹This figure shows a computer with Von Neumann architecture. This kind of computer has a single memory that stores both the program to be executed and the data it is to be executed on, as opposed to the 'Harvard architecture', which has separate data and instruction memories.

to understand. So, the term computer is a broad umbrella encompassing a number of devices, including our PCs, phones, microcontrollers, such as Arduino, Atmel, Raspberry Pi, etc.

If you find figures 3 and 4 to be more densely populated than Figure 5 with additional components, you will not be wrong. The Raspberry Pi is a System-on-a-Chip (SoC). In a traditional computer, the input and output devices will be integrated with the CPU through a motherboard which has circuits translating the messages between CPU and these peripheral devices. The manufacturer of CPU will not be concerned with the peripheral devices. An SoC, on the other hand, is a complete processing system contained in a single package that not only the CPU and memory, but also these interfaces for peripheral devices, the input and output ports, and secondary storage devices. This is captured for the Raspberry Pi 3 in Figure 6.

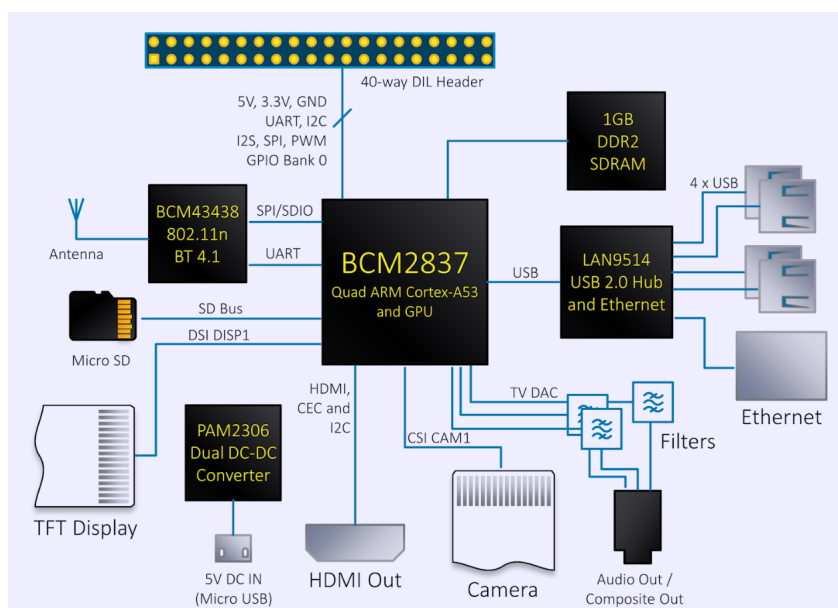


Figure 6: BCM2837 Connections on Raspberry Pi 3

- (a) **State the IC number and purpose of every IC (black or silver rectangular chip) on the provided Raspberry Pi. (1 pt)**

Hint: You can read the code on the IC and search on the internet to find its purpose.

BCM43438 802.11n: Wireless Connectivity, Bluetooth Integration, Low Power Consumption

BCM2837 Quad ARM Cortex A53: Processing, Graphics and Multimedia Support, Compatibility with ARM Architecture

pam2306 Dual DC-DC: Dual Output Channels, Wide Input Voltage Range, Fixed Output Voltages

LAN9514 usb2.0 and 3.0: Plug-and-Play Support, Power Management and Compact Design, Integrated Ethernet Controller

2.2 Operating System

Similar to your computer, the Raspberry Pi also needs an operating system to function. An operating system manages computer hardware and software resources, and provides common services to applications. There are a number of operating systems available for the Pi, including Windows and Linux variants. We'll be using a Linux distribution called Raspberry Pi OS, which is also the recommended OS. The OS for the Pi is stored on the SD card, and so the card serves the same purpose as the hard disk drive on your computer. Information about the OS can be located at <https://www.raspberrypi.com/documentation/computers/os.html>

2.3 Setting up the Pi

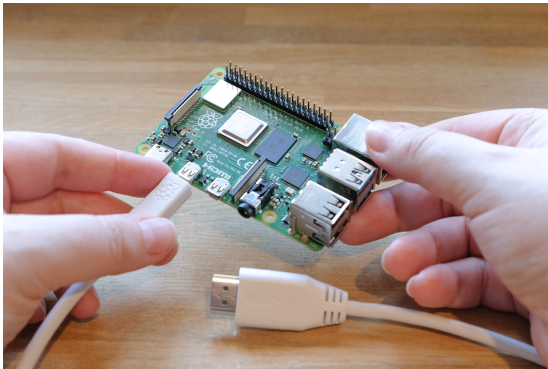
Follow the steps in Figure 7 and Figure 8 to setup the Pi before turning it on. The OS has already been installed and configured for you by your RAs²(Thank them, it is a time-consuming process).



Python

Python 3 is installed by default on Raspberry Pi OS and the “Thonny” IDE is recommended for development. Try writing a simple Python program to test the environment. Instructions to install any additional Python packages are located at <https://www.raspberrypi.com/documentation/computers/os.html#python-on-raspberry-pi>

²If you're setting up your own Raspberry Pi, you can follow the provided instructions in the 'Getting Started' section of the Raspberry Pi documentation (<https://www.raspberrypi.com/documentation/computers/getting-started.html>).



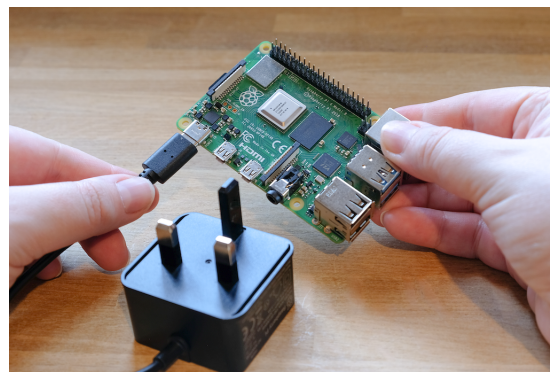
(a) Connect the monitor on your table to your Pi's HDMI port through the HDMI-VGA (μ HDMI-HDMI-VGA) converter.



(b) Connect a keyboard and a mouse to the USB ports of the Pi.



(c) Verify that an SD card is present in the SD card slot.



(d) Connect the power adapter to the Pi's mini-USB/USB-C port.

Figure 7: Setting up the Raspberry Pi.



Figure 8: If you need internet, you can disconnect the ethernet cable from the lab CPU and connect it to the Pi.

2.4 General purpose Input and Output (GPIO)

The marked difference between your PC and Pi is the availability of GPIO pins on the Pi. These pins, as shown in Figure 10 and Figure 9, are an allowance provided by the manufacturers to interface the Pi with other devices. You can also access a handy reference of pin layout on the Pi by opening a terminal window in the OS and running the command `pinout`. This tool is provided by the GPIO Zero Python library, which is installed by default in the OS.

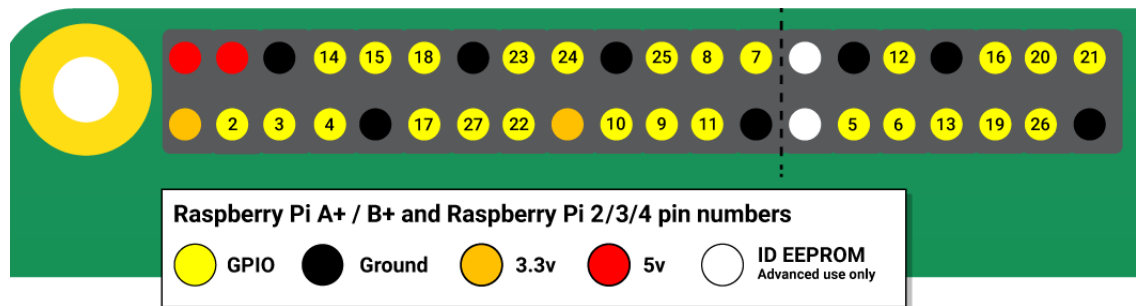


Figure 9: GPIO layout

There are 40 pins, out of which 12 are power or ground pins while the rest are general purpose I/O pins. Each pin acts like a voltage source.



The power pins are at a constant voltage of either 5V or 3.3V, while the ground pins are at a constant voltage of 0V.

The GPIO pins can either be used for input or output. You can set the function of each of these pins as input or output in your Python code.



As an output, a pin will act as a voltage source outputting a voltage of 3.3V when its value is set as HIGH in your code and 0V when set as LOW. So, its voltage value can be switched between the two values through code.

You can already imagine that this can be used to turn on or off any device connected to that pin through software.



A pin set as input will not be provided any power from the Pi internally and it will act like the red voltage probe of a DMM, reading the voltage between this pin and ground. If the read voltage value is close to 3.3V, it will appear as HIGH in our code and values close to 0V will appear as LOW.

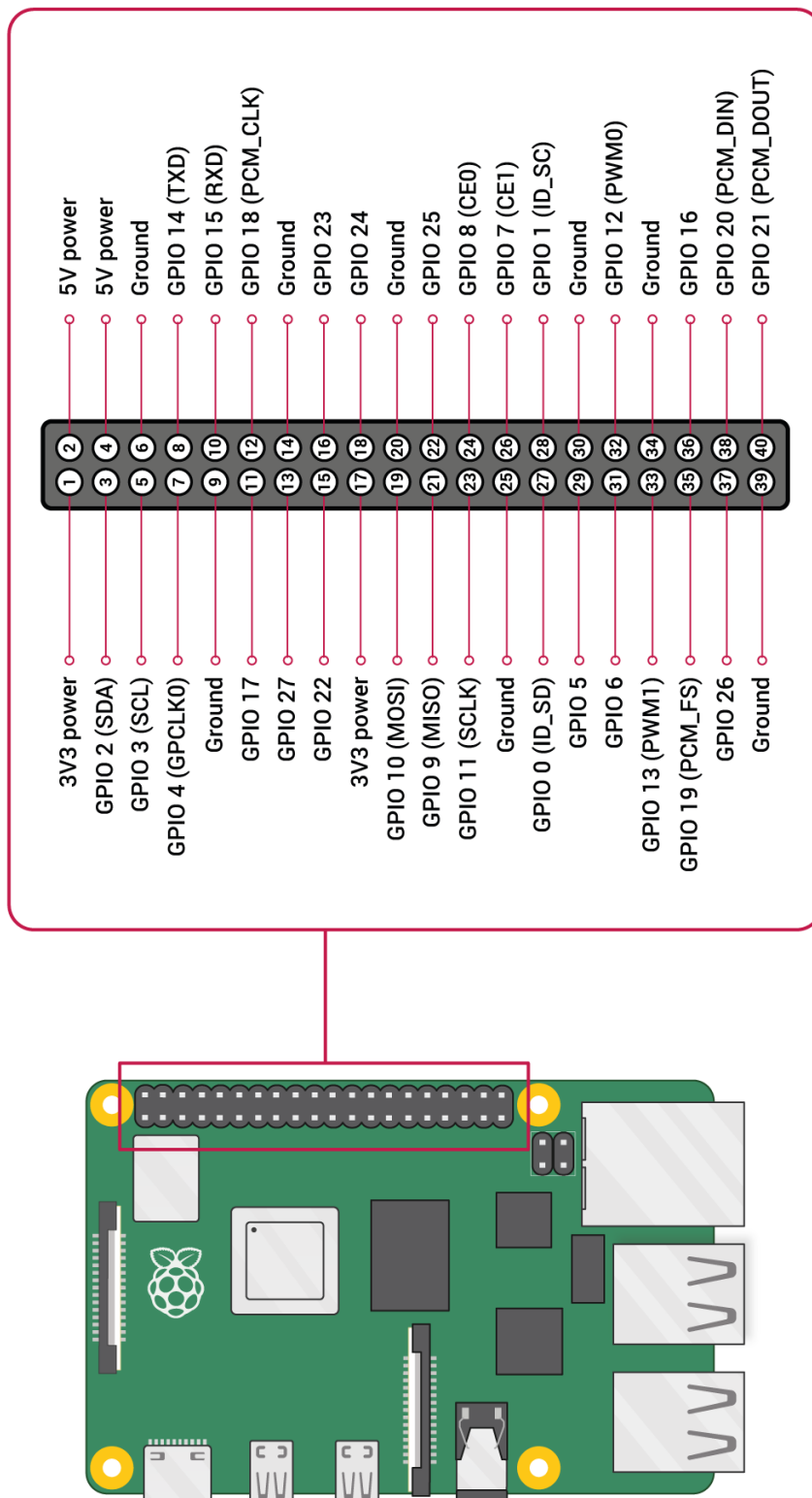


Figure 10: GPIO pins. **Note the number inside the circles is different from the number appearing in the pin name. Number inside the circle is the physical location of the pin on the Pi, while the number in its name is what you'll use to refer to it in your Python code.**



Connecting any pin to a voltage level higher than 3.3V or lower than 0V will DAMAGE the Pi. So, if you need to interface 5V devices to the Pi then you may have to use some level shifting circuit.

Almost all I/O pins have alternative functions too. They can be used to communicate with other devices using communication protocols such as UART, SPI or I2C³.

2.5 Logic Levels

The previous section stated that voltage values close to 3.3V will be viewed as HIGH by the Pi. *How close?* or *What is the voltage level of a 'HIGH' or 'LOW' state of a binary digital system?* The answer to this question will also help us, finally, answer the question: "Why are digital systems resilient to noise?"

It is in fact the job of the digital system designer to associate voltages with the two states of a binary system. For instance, the Raspberry Pi designers have decided that 3.3V is 'HIGH' and 0V is 'LOW'. But then what about 3.2V? Since it is close to 3.3V, it seems reasonable to interpret it as 'HIGH' too. Should we interpret 1.5V as 'HIGH' or 'LOW'?



In order to avoid confusion, it is a standard practice for system designers to define HIGH zones, LOW zones, and forbidden zones of voltages (voltage values in this region are neither considered HIGH or LOW).



Zones defined for inputs to a digital device are different from the zones defined for outputs of a digital device. See the left and right blue blocks in Figure 11.

Consider two digital devices connected to each other, e.g. a GPIO pin of your Raspberry Pi is sending information to your friend's Pi. Your Pi will be called the driver, while your friend's Pi is the receiver. Say, you set the pin on your Pi to a 'HIGH' bit in your code. Your Pi, the driver, will make sure that it generates and transmits a voltage between the voltage level assigned to 'HIGH', i.e. 3.3V, and V_{OH} , e.g. 2.5V to the receiver. It is possible for the voltage to change and get corrupted during transfer, e.g. the wire acts like a resistor and the voltage drops to 3V. The receiver Pi will compare the received voltage value at its pin against its reference values, i.e. is the

³These special communication protocols allow us to transfer large number of bits over fewer wires compared to parallel communication.

⁴Source: Harris, D., & Harris, S. (2010). Digital design and computer architecture. Morgan Kaufmann.

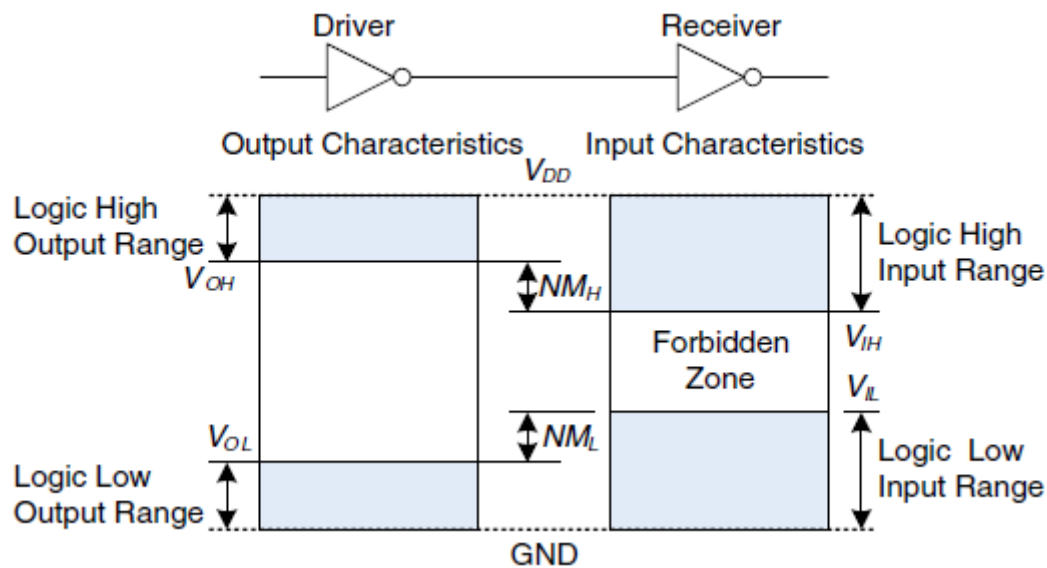


Figure 11: Voltage levels for digital systems⁴. Top blue regions are HIGH zones, the bottom blue regions are LOW zones, and the white region is forbidden. Information about these zones is provided by the manufacturers of digital systems in accompanying datasheets.

received voltage between 3.3V and V_{IH} , e.g. 2V? If yes, then it will consider it as having received a 'HIGH' bit.

Note that V_{IH} , the input threshold, is lower than V_{OH} , the output threshold. So, even if our voltage drops, it is still considered a 'HIGH' bit. This idea is the source of the inherent resiliency to noise in digital systems.

Datasheets of devices will list the zones using the parameters defined below (V_{DD} is the operating voltage of the device):



V_{OH}	When driver has to output 'HIGH', it's guaranteed to generate voltage between V_{OH} and V_{DD} .
V_{OL}	When driver has to output 'LOW', it's guaranteed to generate voltage between GND and V_{OL} .
V_{IH}	If receiver gets an input voltage between V_{IH} and V_{DD} , it will interpret it as 'HIGH'.
V_{IL}	If receiver gets an input voltage between GND and V_{IL} , it will interpret it as 'LOW'.
	In the forbidden zone, the behavior of digital device is unpredictable.



WARNING from MANUFACTURER: “If you follow the instructions, then messing about with the GPIO is safe and fun. Randomly plugging wires and power sources into your Pi, however, may kill it. Bad things can also happen if you try to connect things to your Pi that use a lot of power; LEDs are fine, motors are not.”

Let’s make this warning more concrete so that we always know what is safe to connect to the Pi.

- (b) **Look up the values of V_{OH} , V_{OL} , V_{IH} , and V_{IL} from the hardware specifications of Raspberry Pi** (<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#voltage-specifications>). **(1 pt)**

V_{OH} : 2.6 V V_{OL} : 0.4V
 V_{IH} : 2.0 V V_{IL} : 0.8V

- (c) At the bottom of these voltage specifications table, you’ll find two current values. **State them below along with your Raspberry Pi Model: (1pt)**

Model: Pi 4, Model B

Default Drive Strength: 4mA

Maximum Drive Strength: 8mA

Read the section on ‘GPIO Pads Control’ in the documentation (<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#gpio-pads-control>) till ‘What is a safe current?’ to answer the following questions:

- (d) From reading this section, you’ll have discovered that we have the ability to set the drive strength of any pin on the Raspberry Pi. **If the drive strength of a GPIO pin is set to 8 mA, what does it mean? (1 pt)**

Hint: The answer is directly provided in the reading.

GPIO drive strengths do not indicate a maximum current, but a maximum current under which the pad will still meet the specification. You should set the GPIO drive strengths to match the device being attached in order for the device to work correctly.

- (e) **What is maximum safe value of current to flow through any GPIO pin of your Raspberry Pi model?** (1 pt)

(e) 16mA

- (f) **If you don't set the drive strength of a GPIO pin on your Raspberry Pi, what will be the current it provides at that pin without the voltage dropping?** (1 pt)

(f) 4 mA

- (g) **What will happen if I try to connect different devices to all my GPIO pins and draw the maximum amount of current [the value from (e)] from all pins?** (1 pt)

The Raspberry Pi 3.3V supply was designed with a maximum current of ~3mA per GPIO pin. If you load each pin with 16mA, the total current is 272mA. The 3.3V supply will collapse under that level of load.



3 Lighting an LED

Your first task is to turn an LED on/off from Raspberry Pi. Connect an LED between one of the GPIO pins (the pins that are not marked as power or GND) and a GND pin of the Pi. One possible attachment is shown in Figure 12. Don't forget the resistor and make the circuit on a breadboard.

- (a) What will be the value of the resistor you'll use, if you want current equal to the default drive strength [2(f)] to flow through the LED? Show your calculation. (2 pt)

$$R = V/I$$

$$R = (3.3 - 2) / 0.004$$

$$R = 325 \text{ ohms}$$

Resistor available: 330 ohms

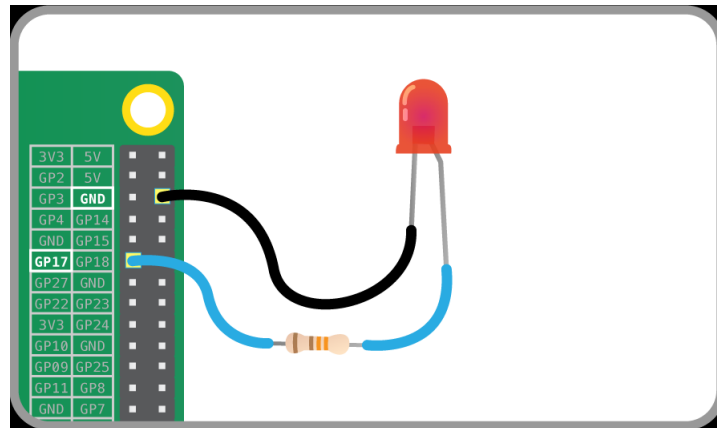


Figure 12: Connecting LED to Pi

The LED will turn on when we set the relevant GPIO pin to HIGH. When we do so, the Pi will turn the pin into a voltage source of 3.3 V and the circuit will be complete. If we set the GPIO pin to LOW, the potential difference between two terminals of the LED will be zero and the LED will remain unlit.

We can set the value of the GPIO pin to HIGH or LOW using Python. Assuming that the LED is connected to GPIO 17 (This is pin number as shown in Figure 9), we can use the following code:

```
1 from gpiozero import LED
2 from signal import pause
3 light = LED(17)
4 light.on()
5 pause()
```

Listing 1: LED on

Let's go through this code line by line.

- `gpiozero` is the library that makes it easy to control a number of devices using the Pi. It provides built-in functions for a number of common devices. The first statement imports

LED class of the GPIOZero library. We can also import the entire library if we simply write `import gpiozero`.

- The second statement similarly imports the function `pause` from the `signal` library.
- The third statement tells the Pi that an LED is connected on GPIO17 and gives the name `light` to that LED. This is creating an instance of the LED class. If we had multiple LEDs, we could create multiple instances, giving each of them their own name, and all of them will inherit the same methods of the LED class.
- The fourth statement calls on the `on()` method⁵ of the LED, which outputs a HIGH on GPIO pin by default.
- The fifth statement pauses execution of the program at the current statement till a key press from the keyboard. This ensures that you can see the program output, otherwise the GPIO pins may reset after program completion and you may not be able to see the LED turning on.
- Other LED methods are `off()`, `toggle()`, and `blink()`. Try them.



You can read about the LED class and other classes in the `gpiozero` documentation at <https://gpiozero.readthedocs.io/en/stable/>.

- (b) **Write a program to make LED blink once every two seconds.** You cannot use the `blink()` method of the LED class. Provide your code in the box below. **(3 pt)**



It may be helpful to use `sleep()` method from `time` library. The statement `sleep(1)` introduces a delay of 1 second in the program.

```
from gpiozer import LED
from time import sleep
from signal import pause
light=LED(17)
while True:
    light.on()
    sleep(1)
    light.off()
    sleep(2)
```

⁵A method is another name for a function, when function is part of a class of related functions.

**Checkpoint**

Recall that we controlled the brightness of an LED using PWM in the previous project, i.e. by adjusting the duty cycle of a square wave that turns the LED on and off at a high frequency.

The PWMLED class from gpiozero library can control the brightness of an LED using PWM. It accepts a value between 0 and 1, where 1 indicates maximum brightness. For example, the following code will light up the LED connected to GPIO17 as before at half the maximum brightness:



```
1 from gpiozero import PWMLED
2 from signal import pause
3 light = PWMLED(17)
4 light.value = 0.5
5 pause()
```

Listing 2: Varying brightness of LED

- (c) **Write a program that fades out an LED by gradually decreasing the brightness of the LED.** You have to write your own code to accomplish this and provide the code in the box below. **(3 pt)**

```
from gpiozero import PWLED
from time import sleep
from signal import pause

light=PWLED(17)
light.on()
while True:
    for i in range(100,0,-1):
        light.value=i/100
        sleep(0.001)
```

**Checkpoint**

- (d) **Use three LEDs, preferably red, yellow, and green, and Write a program that creates a traffic light running in a loop. Attach your code on a separate sheet with your submission. (5 pt)**



Don't forget that each LED will require its own current limiting resistor.



If you're using the same GND pin of the Pi for all your LEDs, remember that the current through that pin will be equal to the sum of currents through all the LEDs. Is that value greater than the maximum current rating of a Raspberry Pi pin?

✓ Checkpoint

- (e) **(Bonus) Build the same traffic light, using a single RGB LED and the RGBLED class. Attach your properly commented code with your submission. The details of the class can be looked up in the gpiozero documentation. (3 pt)**



An RGB LED is a single package that contains three LEDs inside it – a red one, a green one, and a blue one. By adjusting the brightness of three colors, we can generate any color. This LED thus has 4 legs, a common cathode and 3 anodes as shown in Figure 13. We can adjust the brightness of each color using PWM.

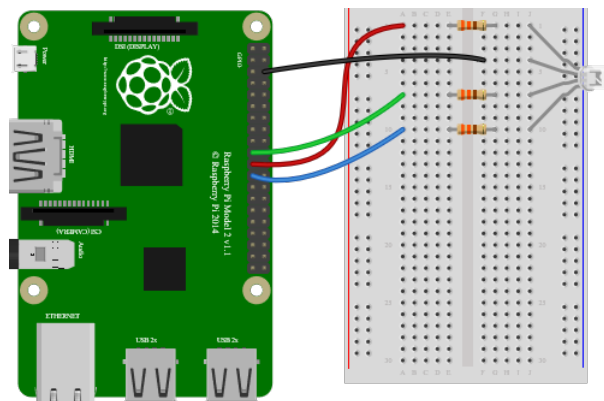


Figure 13: RGB LED

Habib University
Dhanani School of Science and Engineering
Introduction to Electrical and Computer Engineering Lab
Project 2: Digital I - Interacting with physical world using Raspberry Pi

Name: _____

ID: _____

Marks Distribution:

	LR4 Data Collection	LR10 Analysis/ Calculations	LR11 Design	LR12 Equipment Handling (EH)
Task 2a	/1			/2
Task 2b	/1			
Task 2c	/1			
Task 2d		/1		
Task 2e	/1			
Task 2f		/1		
Task 2g		/1		
Task 3a			/2	
Task 3b			/3	
Task 3c			/3	
Task 3d			/5	
Total: 22	/4	/3	/13	/2
CLO	CLO 2	CLO 1	CLO 3	CLO 2

CLO	Total Points	Points Obtained
1	3	
2	6	
3	13	
Total	22	

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.